L06: Nested Conditionals

Security Clearance Levels

Neon City Cyber Academy

Module 2: Control Structures

👥 Your Team

Emily Chen - *Handler*

> "Sometimes one decision isn't enough. You need decisions INSIDE decisions. That's nested conditionals."

Cole Martinez - *Tech Lead*

> "Think of it like security checkpoints. First check: Do you have a badge? Second check: What level is your badge?"

🔄 Review: Simple if/else

```python
age = 20

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

One decision, two outcomes.

But what if we need MORE complexity?

🏢 The Security Checkpoint

```
┌──────────────────┐
│  Have badge?     │
└────────┬─────────┘
   NO    │    YES
         ▼
    ┌──────────────┐
    │  Level?      │
    └──────┬───────┘
       ┌───┴────┐
  GOLD │        │SILVER
```

Two-level decision tree!

🎯 Nested if: Basic Pattern

```python
has_badge = True
level = "gold"

if has_badge:
    if level == "gold":
        print("Full access")
    else:
        print("Limited access")
else:
    print("No access - badge required")
```

An if statement INSIDE another if statement!

💬 Emily's Nesting Guide:

> "Nested conditionals are like folders inside folders:"

```
if (outer condition):
    if (inner condition):
        # Innermost code
    else:
        # Inner else
else:
    # Outer else
```

> "Each level adds another layer of decision-making."

📊 Indentation Levels

```
if condition1:                 # Level 0 indent
    print("A")                 # Level 1 indent
    if condition2:             # Level 1 indent
        print("B")             # Level 2 indent
    else:                      # Level 1 indent
        print("C")             # Level 2 indent
else:                          # Level 0 indent
    print("D")                 # Level 1 indent
```

Each nested level = +4 spaces (1 tab)

🔐 Example: Multi-Level Security

```
rank = input("Rank: ")
clearance = input("Clearance: ")

if rank == "officer":
    if clearance == "top_secret":
        print("Access: All files")
    elif clearance == "secret":
        print("Access: Classified files")
    else:
        print("Access: Public files only")
else:
    print("Access denied - officers only")
```

🎮 Example: Game Difficulty

```python
mode = input("Game mode (solo/team): ")
level = int(input("Level (1-3): "))

if mode == "solo":
    if level == 1:
        enemies = 5
    elif level == 2:
        enemies = 10
    else:
        enemies = 15
else:  # team mode
    if level == 1:
        enemies = 10
    elif level == 2:
        enemies = 20
    else:
        enemies = 30

print(f"Enemies to fight: {enemies}")
```

💬 Cole's Structure Tip:

> "Each branch can have its own sub-decisions:"

```
if role == "admin":
    # Admin path
    if action == "delete":
        confirm_delete()
    elif action == "create":
        confirm_create()
else:
    # Regular user path
    if premium:
        show_premium_features()
    else:
        show_basic_features()
```

🌡️ Example: Weather Advisor

```python
weather = input("Weather (sunny/rainy): ")
temperature = int(input("Temperature: "))

if weather == "sunny":
    if temperature > 30:
        print("Hot & sunny - sunscreen!")
    elif temperature > 20:
        print("Nice day - enjoy!")
    else:
        print("Cool but sunny")
else:  # rainy
    if temperature < 10:
        print("Cold rain - stay inside!")
    else:
        print("Rainy - bring umbrella")
```

🎯 Three-Level Nesting

```python
has_account = True
verified = True
subscription = "premium"

if has_account:
    if verified:
        if subscription == "premium":
            print("Full access")
        else:
            print("Basic access")
    else:
        print("Please verify email")
else:
    print("Please create account")
```

Three levels deep! Use sparingly.

⚠️ When NOT to Nest

```python
# ❌ TOO COMPLEX (4 levels!)
if a:
    if b:
        if c:
            if d:
                print("Hard to read!")
```

Solution: Use `and` operator instead!

```python
# ✅ BETTER
if a and b and c and d:
    print("Much clearer!")
```

💬 Emily's Refactoring Rule:

> "If you have more than 2-3 levels of nesting, STOP and refactor:"

Option 1: Combine with `and` / `or`

Option 2: Use early returns (functions)

Option 3: Break into smaller functions

> "Deep nesting = hard-to-read code. Keep it flat when possible!"

🔄 Nested vs Flat Comparison

```python
# Nested (harder to read)
if user_exists:
    if password_correct:
        if email_verified:
            login()

# Flat (easier to read)
if user_exists and password_correct and email_verified:
    login()
```

Flat is better when conditions are independent!

🎯 Challenge: ATM Machine

```
balance = 1000
card_inserted = True
pin = input("PIN: ")

if card_inserted:
    if pin == "1234":
        action = input("Withdraw/Deposit: ")
        if action.lower() == "withdraw":
            amount = int(input("Amount: "))
            if amount <= balance:
                balance -= amount
                print(f"Success! New balance: ${balance}")
            else:
                print("Insufficient funds")
        elif action.lower() == "deposit":
            amount = int(input("Amount: "))
            balance += amount
            print(f"Deposited! New balance: ${balance}")
    else:
        print("Incorrect PIN")
else:
    print("Please insert card")
```

🏆 Example: Student Grading System

```
attendance = int(input("Attendance %: "))
exam_score = int(input("Exam score: "))

if attendance >= 75:  # Minimum attendance
    if exam_score >= 90:
        grade = "A"
        bonus = 5
    elif exam_score >= 80:
        grade = "B"
        bonus = 3
    elif exam_score >= 70:
        grade = "C"
        bonus = 0
    else:
        grade = "D"
        bonus = 0
else:  # Poor attendance
    if exam_score >= 90:
        grade = "B"  # Penalty for attendance
    else:
        grade = "F"
    bonus = 0

print(f"Grade: {grade}, Bonus: +{bonus}")
```

🐞 Common Nesting Errors

Error #1: Lost in indentation

```
if a:
    if b:
        print("B")
    else:  # Which if does this belong to?
        print("Not B")
```

Fix: Comment which block closures belong to!

🐞 Error #2: Unreachable Code

```
if age >= 18:
    print("Adult")
    if age >= 21:  # This is inside adult block
        print("Can drink")
else:
    if age >= 21:  # ❌ UNREACHABLE!
        print("This never runs")
```

If age < 18, we're in else. Can't be ≥21!

✅ Lesson Complete!

You Mastered:

- ✅ Nested if statements (if inside if)
- ✅ Multi-level decision trees
- ✅ Proper indentation for nested blocks
- ✅ When to nest vs when to flatten
- ✅ Security clearance level patterns
- ✅ Complex real-world examples

🏆 Your Assignment

Create a **Movie Ticket System:**

Requirements:

1. Ask age
2. Ask if student (yes/no)
3. Ask if VIP member (yes/no)

Pricing:

- Under 12: $5
- Student (12-18): $7
- Adult (18+): $10
- VIP gets 20% discount on any price

🌆 Congratulations!

You've completed **Module 2: Control Structures**!

Next: Module 3 - Functions & Modules

Build reusable code blocks and organize like a pro!